

Training FLOPs, Inference Compute & Sparse MoE.

Three faces of “compute.” First we derive the $6ND$ training-FLOP law that underpins all scaling work. Then we meet a second, orthogonal axis — *inference-time* compute, where you buy accuracy by generating more tokens. Finally, sparse Mixture-of-Experts breaks the link between *how many* parameters a model has and *how many* FLOPs each token costs. Every number below is computed in full, nothing summarised or deferred.

THE EQUATION

$$C_{\text{train}} \approx 6ND \quad \underbrace{2N}_{\text{fwd}} + \underbrace{4N}_{\text{bwd}} = 6N \text{ FLOPs/param-token} \quad \text{MoE active frac} = \frac{k}{E}$$

Worked example. Training FLOPs for $N=70\text{e9}$, $D=1.4\text{e12}$; inference cost of generating 1/100/1000 tokens; and an $E=8$, top-2 MoE showing a 75% FLOP saving versus a dense model of the same total size.

Topic 2 of 2 in this module · companion to the course page · v1.0

Contents

1	Deriving the training-FLOP law $C \approx 6ND$	2
2	A second axis: inference-time compute	3
3	Sparse Mixture-of-Experts: decoupling capacity from compute	4
4	Properties / consequences that matter	4
5	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 Deriving the training-FLOP law $C \approx 6ND$

1.1 Forward pass: $\approx 2N$ FLOPs per token

Almost all of a transformer's compute lives in matrix multiplications: every weight w in the network is used in an expression $w \cdot x$ that is accumulated into an output. Processing one token through the whole network applies each of the N parameters essentially once, and each application is one multiply plus one add into the running sum — two floating-point operations. Hence the forward pass costs

$$\text{fwd} \approx 2N \text{ FLOPs per token.}$$

1.2 Backward pass: $\approx 2 \times$ the forward

Backpropagation computes, for each weight matrix, two products of the same size as the forward multiply: the gradient w.r.t. the *input* (to pass error to the previous layer) and the gradient w.r.t. the *weight* (to update it). Each is the same FLOP cost as the forward multiply, so the backward pass costs about twice the forward:

$$\text{bwd} \approx 2 \times (\text{fwd}) = 4N \text{ FLOPs per token.}$$

Adding forward and backward gives $\approx 6N$ FLOPs per parameter per token. Multiplying by D tokens (each seen once in one epoch):

$$C_{\text{train}} \approx \underbrace{2N}_{\text{fwd}} + \underbrace{4N}_{\text{bwd}} \text{ per token} \times D \text{ tokens} = 6ND \quad (1)$$

Think “six FLOPs per parameter, per token.” Two to look at the token going forward, four to learn from it coming back. The whole budget is then just that constant times how many parameters you have (N) times how many tokens you show (D). It is this clean product form — a budget you can split between N and D — that made the Chinchilla optimisation in Topic 1 possible.

1.3 Worked number

For Chinchilla’s allocation $N = 70\text{e}9$, $D = 1.4\text{e}12$:

$$C_{\text{train}} = 6 \times (70 \times 10^9) \times (1.4 \times 10^{12}) = 6 \times 9.8 \times 10^{22} = 5.880 \times 10^{23} \text{ FLOPs.}$$

That is 588 zettaFLOPs — the headline training cost of the model from Topic 1, now derived rather than asserted.

2 A second axis: inference-time compute

Training FLOPs ($6ND$) are a *one-time* cost paid before deployment. There is a second, *orthogonal* compute axis paid at *every query*: how much computation you spend *generating* an answer. The model weights are frozen, but you can still “think harder” by:

- **Chain-of-thought:** generate intermediate reasoning tokens before the final answer.
- **Sampling + verification:** draw many candidate answers and pick the best with a verifier or majority vote (self-consistency).
- **Search:** expand and score a tree of partial solutions (e.g. tree-of-thought).

Each generated token in a dense model costs one forward pass, $\approx 2N$ FLOPs. So generating G tokens costs

$$C_{\text{infer}} \approx 2NG \quad (2)$$

For $N = 70\text{e}9$, the per-token forward cost is $2N = 1.4 \times 10^{11}$ FLOPs. Spending more tokens trades compute for accuracy on hard problems:

G (tokens generated)	$C_{\text{infer}} = 2NG$ (FLOPs)	regime
1	1.400×10^{11}	direct answer
100	1.400×10^{13}	short chain-of-thought
1000	1.400×10^{14}	long reasoning / search

Training scale ($6ND$) and inference scale ($2NG$) are *different knobs*. A modest model that is allowed to reason for 1000 tokens can outperform a larger model answering in one shot on hard tasks — you bought capability with test-time compute instead of training

compute. This is the foundation of “reasoning” models in the GPT-4 era.

3 Sparse Mixture-of-Experts: decoupling capacity from compute

A dense feed-forward layer runs *every* token through *all* its parameters. A Mixture-of-Experts (MoE) layer instead holds E parallel “expert” sub-networks and a small *gate* that, for each token, selects only the top- k experts to run. Capacity (total parameters) grows with E , but compute per token grows only with k .

$$\text{active-param fraction} = \frac{k}{E}, \quad \text{FLOP saving vs. dense same-size} = 1 - \frac{k}{E} \quad (3)$$

3.1 Worked example: $E = 8$ experts, top-2 routing

With $E = 8$ and $k = 2$, each token is processed by only 2 of the 8 experts:

$$\text{active fraction} = \frac{k}{E} = \frac{2}{8} = 0.25, \quad \text{FLOP saving} = 1 - \frac{2}{8} = 0.75 = 75\%.$$

Concretely, if each expert holds $P_e = 7 \times 10^9$ parameters, the experts together hold $EP_e = 8 \times 7e9 = 5.6 \times 10^{10}$ parameters, but only $kP_e = 2 \times 7e9 = 1.4 \times 10^{10}$ are active per token. A *dense* model with the same 5.6×10^{10} parameters would run all of them on every token; the MoE runs a quarter, a $4\times$ FLOP reduction (the 75% saving).

Routing	active frac k/E	FLOP saving $1 - k/E$	active params ($P_e=7e9$)
top-1 of 8	0.125	87.5%	7.0×10^9
top-2 of 8	0.250	75.0%	1.4×10^{10}
top-4 of 8	0.500	50.0%	2.8×10^{10}
dense ($k=8$)	1.000	0.0%	5.6×10^{10}

The heatmap shades the active-parameter fraction; top-2 lights up only a quarter of the model (capped at indigo!80 for the dense baseline).

	top-1	top-2	top-4	dense
k/E	0.125	0.250	0.500	1.000

4 Properties / consequences that matter

1. **6ND counts only matmuls.** Attention’s $O(\text{seq}^2)$ term, layer norms, softmaxes, and embeddings are lower order at scale, so 6ND is an excellent first-order estimate — the basis of the Chinchilla budget in Topic 1.
2. **Two compute axes.** Total cost is training (6ND, paid once) *plus* inference (2NG per query, paid forever). Optimising one ignores the other (cf. the inference-optimal pitfall in Topic 1).

3. **MoE decouples N from FLOPs.** An MoE can advertise a huge *total* parameter count while costing only k/E of the FLOPs of an equally large dense model — more knowledge per FLOP.
4. **FLOPs scale with k , not E .** Adding experts (raising E) grows capacity essentially for free in compute; only k (experts per token) moves the FLOP bill.

“MoE parameters are free.” The *FLOP* saving is real, but every expert’s weights must still live in memory and move across the interconnect during routing. A top-2-of-8 model has $4\times$ the memory footprint of its active slice and incurs all-to-all communication plus load-balancing overhead. So MoE trades a 75% FLOP cut for higher memory and bandwidth cost — a win only when compute, not memory, is the bottleneck.

5 Where this sits in the track

This topic supplies the $6ND$ law that Topic 1 (Chinchilla) takes as its compute constraint, and it opens the two frontiers that define the GPT-4 era: *test-time compute* (reasoning models) and *sparsity* (MoE). The full Mixture-of-Experts treatment — gating networks, the load-balancing auxiliary loss, expert parallelism, and capacity factors — is developed in Module 7.1.

A Reproducible (NumPy)

Inputs: $N = 70\text{e}9$, $D = 1.4\text{e}12$; generation lengths $G \in \{1, 100, 1000\}$; MoE $E = 8$, $k \in \{1, 2, 4, 8\}$, per-expert $P_e = 7\text{e}9$. The snippet reproduces every FLOP value, fraction, and saving above.

```
import numpy as np
N, D = 70e9, 1.4e12
print("train C=6ND = %.3e" % (6*N*D))          # 5.880e+23
for G in [1, 100, 1000]:
    print("G=%d infer=2NG = %.3e" % (G, 2*N*G))
# G=1    infer=2NG = 1.400e+11
# G=100  infer=2NG = 1.400e+13
# G=1000 infer=2NG = 1.400e+14
E, Pe = 8, 7e9
for k in [1, 2, 4, 8]:
    frac = k/E
    print("k=%d active=%.3f saving=%.1f%% active_params=%.2e"
          % (k, frac, (1-frac)*100, k*Pe))
# k=1 active=0.125 saving=87.5% active_params=7.00e+09
# k=2 active=0.250 saving=75.0% active_params=1.40e+10
# k=4 active=0.500 saving=50.0% active_params=2.80e+10
# k=8 active=1.000 saving=0.0%  active_params=5.60e+10
```